PRODUCT X CAMPAIGN

Uplift Modeling Framework

Identifying Customers Who Generate Incremental Net Revenue When Offered a 15% Discount

Data Science Team · Sr. Data Scientist Case Study

Why the Campaign Didn't Work

Understanding the four customer types that determine campaign value

✓ Target

Persuadables

Would NOT buy without discount — DO buy with it. These are the only customers who generate true incremental value.

✗ Avoid

Sure Things

Would buy regardless of the discount. Offering them 15% off is pure margin loss with zero volume upside.

— Skip

Lost Causes

Won't buy regardless. Neither the discount nor its absence changes their behavior. Waste of effort.

✗ Avoid

Sleeping Dogs

Might buy without a discount, but the discount actually deters them (rare, but data-driven models catch this).

The Solution: Uplift Modeling

Model the causal effect of the discount per customer — not just their probability of purchase

01

Estimate Individual Treatment Effect

For each customer, estimate:
τ_i = P(buy | discount) − P(buy | no discount)

02

Translate to Net Revenue Uplift

Uplift in revenue = τ_i × price after discount − baseline revenue loss from price cut

03

Target Only Positive-Uplift Customers

Send discount only to customers where predicted incremental net revenue > 0

Data Available & Feature Engineering

Dataset at a Glance

485,473
Total customers

80% / 20%
Treatment / Control split

~0.27% / ~0.22%
Purchase rate: treated / control

33 features
RFM windows: L1M → L36M

CMPGN_NET_REV
Target: net revenue during campaign

Key Engineered Features

- **Recency Score:** Days since last transaction
- **Purchase Velocity:** L1M / L3M / L6M transaction counts
- **Revenue Trend:** L3M net rev ÷ L12M net rev
- **Avg Basket Size:** Gross rev ÷ number of transactions
- **Loyalty Tenure:** Days since ENROLL_DT
- **Revenue per Visit:** Net rev ÷ num transactions

Modeling Approaches

Three methods — each with trade-offs in complexity, interpretability, and data requirements

Recommended Start

Two-Model (S-Learner / T-Learner)

- T-Learner: train Model_T on treated, Model_C on control
- Uplift = P(buy | treated) − P(buy | control)
- Simple to implement with XGBoost or LightGBM
- Risk: each model may learn different feature spaces

Better with Imbalance

Meta-Learner (X-Learner)

- Cross-fits residuals between treated and control groups
- More stable when 80/20 split makes groups differ in size
- Adds one extra modeling layer but improves variance
- Use CausalML or EconML libraries

Most Rigorous

Transformed Outcome / DR-Learner

- Doubly-robust: requires both propensity + outcome model
- Consistent estimator even if one model is misspecified
- Best for production if causal rigor is paramount
- More data-hungry; check sample sizes per segment

Python Implementation Stack

7 of 10 skills — each mapped to a specific modeling phase

01 **pandas** — Data Prep

Load, clean, and merge RFM time-window features; create treatment flag (TARGETED_DISCOUNT > 0).

02 **scikit-learn** — Feature Eng.

Pipeline with ColumnTransformer (OrdinalEncoder + StandardScaler); train/test split stratified on treatment.

03 **xgboost / lightgbm** — Base Learners

Gradient boosting models for P(buy | treated) and P(buy | control) — the T-Learner pair.

04 **causalml / econml** — Uplift Modeling

UpliftTreeClassifier, X-Learner, or DR-Learner wrappers on top of the base learners.

05 **shap** — Explainability

SHAP values per customer to understand which features drive positive vs negative uplift.

06 **matplotlib / seaborn** — Evaluation Viz

Uplift curves, Qini coefficient plots, gain charts — the key evaluation tools for uplift models.

07 **scipy / statsmodels** — Validation

Bootstrap confidence intervals on Qini; t-tests on net revenue uplift in top deciles.

Skill 1–4: T-Learner Uplift — Code Sketch

pandas · scikit-learn · xgboost · causalml

```
import pandas as pd, numpy as np
from
 xgboost import XGBClassifier
from
 causalml.inference.meta import BaseTLearner

# 1. Load & prep  (pandas)
df = pd.read_excel('campaign_data.xlsx')
df['treated'] = (df['TARGETED_DISCOUNT'] > 0).astype(int)

# 2. Feature matrix  (scikit-learn preprocessing)
FEATURES = ['L12M_NUM_TRANS', 'recency_days', 'revenue_trend', ...]  # engineered

# 3. Fit T-Learner  (causalml wrapping xgboost)
learner = BaseTLearner(learner=XGBClassifier(n_estimators=300))
learner.fit(X=df[FEATURES], treatment=df['treated'], y=df['purchased'])

# 4. Predict uplift score for every customer
df['uplift_score'] = learner.predict(X=df[FEATURES]).flatten()
df['send_discount'] = df['uplift_score'] > net_revenue_breakeven_threshold
```

Skill 5–7: Explainability, Evaluation & Validation

shap · matplotlib / seaborn · scipy / statsmodels

```
import shap
import matplotlib.pyplot as plt
from scipy import stats

# Skill 5: SHAP explainability  (on treated-group model)
explainer = shap.TreeExplainer(learner.models_tau[1])  # T-Learner treated model
shap_vals = explainer.shap_values(df_test[FEATURES])
shap.summary_plot(shap_vals, df_test[FEATURES])  # reveals key uplift drivers

# Skill 6: Qini curve  (canonical uplift evaluation)
from causalml.metrics import plot_qini
plot_qini(df_test, outcome_col='purchased', treatment_col='treated')

# Skill 7: Bootstrap CI on net revenue lift in top decile
top_decile = df_test.nlargest(int(0.1 * len(df_test)), 'uplift_score')
t_stat, p_val = stats.ttest_ind(
    top_decile[top_decile.treated==1]['CMPGN_NET_REV'],
    top_decile[top_decile.treated==0]['CMPGN_NET_REV']
)  # Confirms statistical significance of targeting gain
```

Targeting Strategy & Break-Even Economics

Break-Even Threshold = (Price × 15% discount) ÷ P(buy | control)

Send the discount only when: τ_i > Threshold → E[Incremental Net Revenue] > 0

High Uplift Tier

- Positive uplift > threshold
- Send 15% discount (as tested)
- Expected to be net positive
- Monitor CMPGN_NET_REV vs control

Borderline Tier

- Uplift near threshold
- Consider A/B within segment
- Could test smaller discount
- Needs more data to confirm

Do Not Target

- Sure Things & negative uplift
- No discount sent
- Preserves margin
- Largest segment by volume

Summary & Implementation Roadmap

Phase 1 (Week 1–2)

EDA → feature engineering → T-Learner prototype
Validate Qini > 0 before advancing

Phase 2 (Week 3–4)

X-Learner or DR-Learner → SHAP analysis
Calculate break-even threshold per customer

Phase 3 (Week 5+)

Targeted holdout test — top decile vs random 80%
Measure incremental net revenue vs full blast

Success KPIs

- ✓ Qini coefficient > 0
- ✓ Net rev per targeted customer > control
- ✓ Top-decile lift > full-blast lift
- ✓ Conversion rate maintained

The fundamental insight: the campaign's problem is not the offer — it's that the offer was wasted on customers who didn't need it. Uplift modeling fixes the targeting without changing the discount.